

In the world of programming, hashing is a method of storing key-value pairs and this is achieved by making use of HashMaps in Java. In addition to storing key-val pairs, HashMaps are extensively used for solving a range of problems in information technology. By reducing the time complexity of various operations, hashmaps can make problem-solving much quicker when used in combination with different data structures.

What is a HashMap?

A collection based on the map which is used for storing key-value pairs in Java is known as a HashMap. All the keys which are stored in a Hashmap are unique.

Let's see the syntax for how we can create a HashMap in Java:

```
HashMap<Key, Value_mapped_to_the_key> name_of_your_map = new HashMap<>();
```

To give you an idea of HashMap let's consider an example:

Suppose you want to store the roll number of students in a class along with their names then you can easily do so with the help of HashMaps. The name serves as the key for the hashmap and the roll number as the value which can be mapped to the names.

Dave	16
Robin	19
Selena	32
Mark	10

As you can observe there is no ordering of names because hashmaps do not follow any particular ordering for storing the elements.

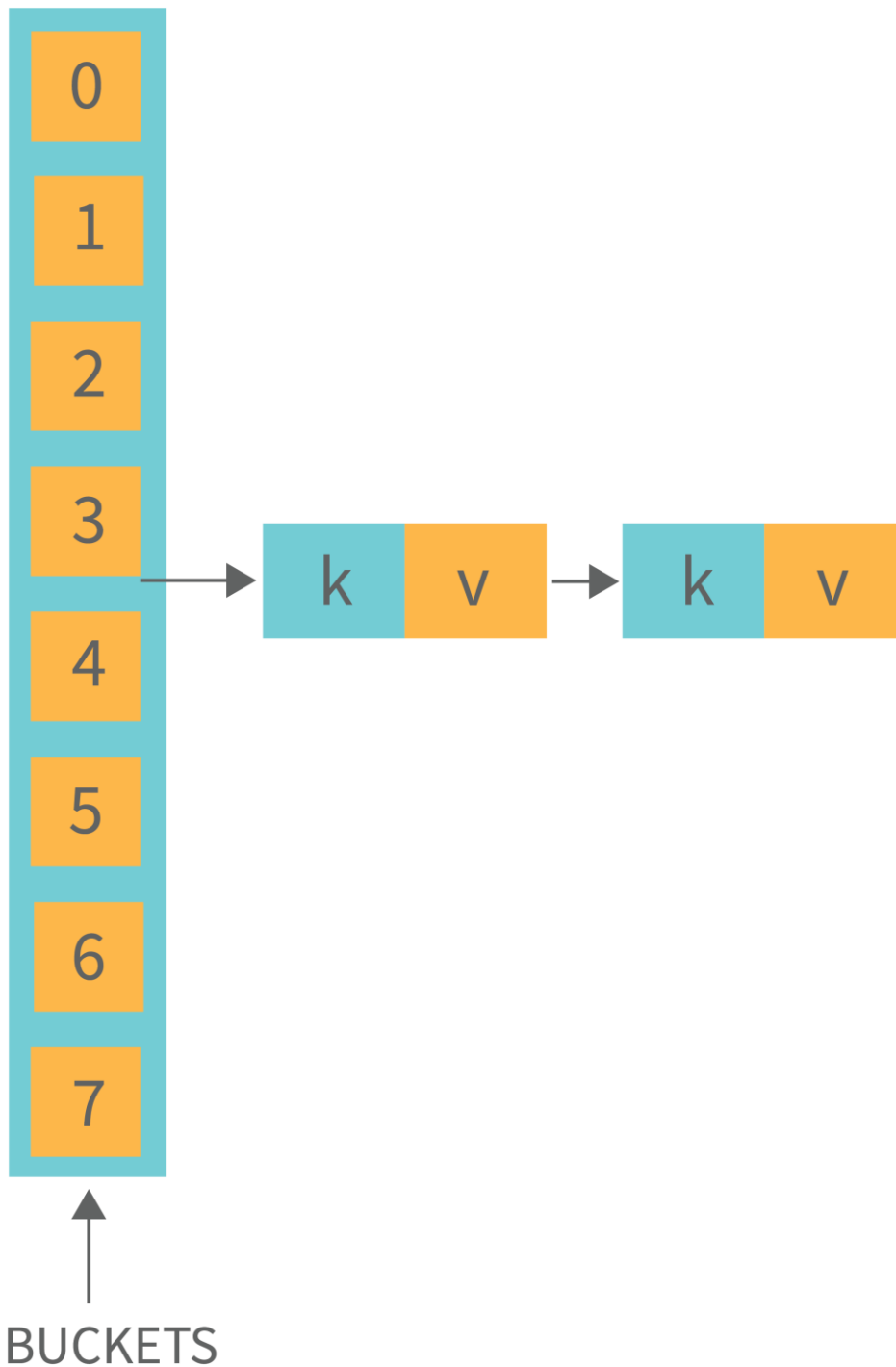
Some important properties of a Hashmap in Java to remember:

- Hashmaps are basically used for storing key-value pairs using a hashtable
- Hashmap stores only unique keys
- There is no particular ordering of elements in a HashMap
- Import java.util.HashMap in your program for using a HashMap

Now in this article, we will be covering the most important and frequently asked **Java HashMap Interview Questions and Answers** to help you ace the interviews. We will cover different varieties of questions that cater to both freshers as well as experienced programmers.

HashMap Interview Questions for Freshers

1. Explain the internal working of a HashMap.



- A hashmap uses a hashtable but what many people don't know is that it is internally created using two data structures namely an array and a linked list. Whenever you declare a hashmap what is going to happen internally is that it will create an array of buckets. The buckets are referred to as nodes or you can say a linked list. These nodes contain mainly:
 - The key,
 - Value
 - Hashcode
 - Address of the next node.
- Now, when you insert values in a key like this:

```
map.put("hashmap implementation in Java",1);
```

Then hashcode will be calculated by the put method. This hashcode will help us to store the key at a particular index. Also, hashcode will make the process of retrieving the values faster.

```
hashcode=hash("hashmap implementation in Java");
```

This hash code is further computed and will generate an index for storing the value. The value of our key will be stored at the index given by the hashcode in the form of a LinkedList.

- It's important to note that Java 8 introduced BST in place of a linked list to make retrieval of the key-value pairs more efficient.

2. Which property of a hashmap is used to find the intersection of two arrays?

- Since a hashmap stores only unique elements, this property of a hashmap can be used to find intersections i.e common elements of two arrays.
- The approach to this problem is: first we will create a hashmap and fill the elements of the first array in it.
- The next step is to iterate over the second array and check whether the element is present in the hashmap or not.
- In this way, we can find all common elements of two arrays in $O(n)$ time complexity.

3. What is collision in HashMap?

- A collision occurs when more than one key generates the same hashCode() value.
- An inefficient hashCode() algorithm causes considerable collisions in a hashmap.
- An increased number of collisions can affect the performance of a hashmap.

4. Discuss the approach for calculating all the equal combinations formed by a pair of numbers in an array using a hashmap i.e The numbers must satisfy these two conditions: $\text{nums}[i] == \text{nums}[j]$ and $i < j$ Answer)

- You all must have studied combinations, so the concept of combinations states that every time we just have to select a pair from n numbers in an array that satisfies the given conditions and the formula for doing so is NCR which is equivalent to $\frac{n*(n-1)}{2}$.
- Now hashmap will be used for calculating the frequency of all the numbers in the array.
- While iterating the array we will store the frequency of every element in a hashmap.
- The final step would be to iterate over the map and apply the formula $\frac{n*(n-1)}{2}$, where n is the frequency of each element in the array.

5. How Does Java's Capacity And Size Of Hashmap Differ?

Sometimes it's confusing for students to understand the difference between capacity is size.

So basically, HashMap's capacity indicates the number of entries it can hold, while its length indicates how many key-value pairs are presently in existence.

6. How can iteration be performed in a HashMap?

```
import java.util.Map;
import java.util.HashMap;

public class example_____iteration
{
    public static void main(String[] arg)
    {
        Map<String,String> INTERVIEW_BIT = new HashMap<String,String>();

        INTERVIEW_BIT.put("hashmap coding interview questions", "AT INTERVIEWBIT");
```

```
INTERVIEW_BIT.put("hashmap programming interview questions", "AT INTERVIEWBIT");

for (Map.Entry<String,String> entry : INTERVIEW_BIT.entrySet())
    System.out.println("Key = " + entry.getKey() +
        ", Value = " + entry.getValue());
}
}
```

Output:

```
Key = hashmap coding interview questions, Value = AT INTERVIEWBIT
Key = hashmap programming interview questions, Value = AT INTERVIEWBIT
```

7. What is the order in which keys are stored in a hashmap?

- There is no particular order for storing keys in a hashmap. All the keys are stored in an unsorted order.
- For storing the elements in a particular order a Treemap can be used in Java but a hashmap does not guarantee the ordering of your elements.

8. Does hashmap allow you to store null values?

Yes, We can store as many null values in a hashmap as we want but only one null key can be stored not more than that.

Let's see a program that depicts how we can store null values in a hashmap.

```
import java.util.*;
import java.io.*;
public class interviewBit {
    public static void main(String[] args)
    {
        HashMap hmap=new HashMap();
        hmap.put(1,"Program to store null value");
        hmap.put(null,"InterviewBit");
        System.out.println(hmap);
    }
}
```

OUTPUT:

```
{null=InterviewBit, 1=Program to store null value}
```

9. Specify whether hashmaps in Java are thread-safe or not?

- The answer is "NO" i.e `java.lang.HashMap` does not support thread-safety. If several threads are altering the HashMap, for example, insertions or removals, then a HashMap should not be shared with other threads.

- If you want to implement thread-safety then you can either use a `ConcurrentHashMap` or by using the `Collections.synchronizedMap()` method.
- When data consistency is crucial, `Collections.synchronizedMap()` should be used, and `ConcurrentHashMap` should be used where the number of reads and write operations to be performed are less.
- Since `Collections.synchronizedMap()` needs every thread to attain a lock on the whole object to accomplish both read and write operations, this results in slow performance.
- With `ConcurrentHashMap`, threads can simultaneously modify sections while maintaining a lock on individual segments.

10. Specify different methods of creating a hashmap in java along with implementation.

Different methods of creating a hashmap are:

1) Constructing a hashmap with default capacity

- **Syntax:**

```
HashMap<String, Integer> InterviewBlt_map1 = new HashMap<String, Integer>();
```

2) Constructing a hashmap with a defined capacity i.e 50 in our example

- **Syntax:**

```
HashMap<String, Integer> InterviewBlt_map2 = new HashMap<String, Integer>(50);
```

3) Specifying load factor along with the capacity

- **Syntax:**

```
HashMap<String, Integer> InterviewBlt_map3= new HashMap<String, Integer>(50, 0.5f);
```

4) By copying another map into our map

- **Syntax:**

```
HashMap<String, Integer> InterviewBlt_map4 = new HashMap<String, Integer>(InterviewBlt_map1);
```

11. Can you store multiple keys with the same value in a hashmap?

- No, multiple keys with the same value can't be stored in a hashmap.
- When you try to insert a new key that is already present in the hashmap then overriding will happen and the old key will be replaced with the new key and a new value.

12. State the differences between a Hashmap and a Hashtable in Java.

HashMap Vs Hashtable

Parameter	HashMap	Hashtable
Synchronization	Hashmap and Hashtable are very similar to each other except for the fact that a hashmap does not allow synchronization	Unlike a Hashmap, synchronization is permitted in a Hashtable
Performance	You won't face any performance issues in a hashmap.	Synchronization in a hashtable might sound like a good idea that a particular element would be thread-safe if there's concurrent access with multiple threads however, the problem with synchronizing every method that allows access to the underlying collection is that it creates performance issues. So you will have performance issues while using a hashtable
Null values	Null values are permitted in a Hashmap i.e you can store as many null values in a hashmap as you want.	Unlike a Hashmap, Null values are not permitted in a hashtable
Null keys	While using a Hashmap you can store only one null key not more than that.	In a Hashtable null keys, as well as null values, are not allowed similar to a ConcurrentHashMap.
Iterators	We can use iterators for looping through the key and value pairs in a Hashmap.	We can use enumerators, as well as iterators, for looping through the key and value pairs in a Hashtable.

13. What is the time complexity in terms of big o notation of pushing and retrieving an element from a hashmap?

- Time complexity is the measure of the number of CPU cycles utilized by a program to execute.
- Hashmap time complexity for pushing and retrieving the elements is the order of 1 i.e $O(1)$ using put and get methods respectively.

HashMap Interview Questions for Experienced

1. What is the maximum number of entries you can store in HashMap?

HashMap does not have a maximum number of entries because when the bucket is full, the keys will be added to a linked list, which can store forever until all the memory you have is consumed.

2. What factors determine the performance of a hashmap?

The performance or efficiency of a hashmap depends basically on two things that are:

- **Capacity** - Capacity is defined as the total number of buckets present in a Hashmap and Sixteen is the default capacity when a Hashmap is created by the user. The capacity can be increased when more key-value pairs are added to the hashmap.
- **Load Factor** - As discussed in the previous point the capacity of the map can be increased but when should the hashmap increase its capacity?

This is decided by the load factor and its default value is seventy-five per cent.

3. How can a hashmap be used to check whether two given arrays are equal or not i.e they contain the same elements or not? Given the arrays can be unsorted.

- A hashmap can be easily used to check whether the given arrays are equal or not.
- First, create a hashmap and using a for loop fill the frequency of each and every element of array 1 in the hashmap.
- Now, make another loop and decrement the frequency of each and every element of the second array in the hashmap.
- In the next step, iterate over the hashmap and check whether all the frequencies are zero or not. Even if any frequency is 1 return false else return true.

4. Distinguish between a hashmap and a TreeMap.

Hashmap	TreeMap
In a hashmap, no ordering of elements is maintained	The ordering, of elements, is maintained in a treemap.
We can store as many null values in a hashmap as we want but only 1 null key can be stored	We cannot store any null values or keys in a treemap.
Hashmap is Fast since no ordering of elements is maintained therefore elements can be inserted and retrieved in constant time.	TreeMap is slow and most of the functions take logarithmic time.
Hashmap uses arrays and LinkedList which forms a hashtable for its implementation internally.	TreeMap is implemented with the help of red-black trees.

5. Which is the best technique to handle collision in a hashmap.

As part of its collision handling, HashMap employs chaining. In chaining, a linked list is used for placing the key-value pairs inserted into the map with the value already present to avoid collision in the map at a bucket location as the newly inserted value is placed in front of the linked list.

6. Define LinkedHashMap in Java

- A linked HashMap is implemented in a similar way to a hashmap but contains a doubly LinkedList for querying through all the key-value pairs.
- LinkedHashMap stores the key-value pair in order, unlike a hashmap.
- You can retrieve the data in the same order as they were inserted in the map i.e the key which was inserted first can be taken out first.

Implementation of LinkedHashMap for understanding how we can insert and retrieve key and value pairs easily:

```
import java.util.LinkedHashMap;
public class InterviewBit {

    public static void main(String[] args) {

        LinkedHashMap m = new LinkedHashMap();

        m.put("Linked", new Integer(1));
        m.put("Map", new Integer(2));

        Object lobj = m.get("Linked");
        System.out.println(lobj);
    }
}
```

The output of the above program will be 1.

7. Distinguish between a Hashmap and ConcurrentHashMap in Java along with the implementation of both.

Hashmap vs ConcurrentHashMap:

1. **ThreadSafe:** One of the most significant differences between both is that a ConcurrentHashMap is synchronized internally and thread-safe which makes it suitable for a multithreaded environment whereas a hashmap is non-synchronized as well non-thread-safe which makes it unsuitable for a multithreaded environment.
2. **Null Keys And Null Values:** As we all know, data is kept in the form of key and value pairs in a hashmap and we can store as many null values in a hashmap as we want but only 1 null key can be stored whereas in a concurrent hashmap null keys, as well as null values, are not permitted.

Implementations

- Hashmap Implementation:

```
import java.util.*;
import java.io.*;
public class interviewBit {
    public static void main(String[] args)
    {
        HashMap hmap=new HashMap();
        hmap.put(91,"Hashmap Implementation");
        hmap.put(92,"in ");
        hmap.put(93,"Java");
        hmap.put(null,"InterviewBit");
        System.out.println(hmap);
    }
}
```

OUTPUT:

```
java -cp /tmp/ZPv88JJ0D2 interviewBit
{null=InterviewBit, 91=Hashmap Implementation, 92=in , 93=Java}
```

- ConcurrentHashMap implementation:

For implementing ConcurrentHashMap we have to import the concurrent package.

```
import java.util.concurrent.ConcurrentHashMap;
public class interviewBit {
    public static void main(String[] args)
    {
        ConcurrentHashMap hmap=new ConcurrentHashMap();
        hmap.put(91,"Hashmap Implementation");
        hmap.put(92,"in ");
        hmap.put(93,"Java");
        hmap.put(null,"InterviewBit");
        System.out.println(hmap);
    }
}
```

```
}  
}
```

OUTPUT:

```
java -cp /tmp/ZPv88JJ0D2 interviewBit  
Exception in thread "main" java.lang.NullPointerException  
at java.base/java.util.concurrent.ConcurrentHashMap.putVal(ConcurrentHashMap.java:1011)  
at java.base/java.util.concurrent.ConcurrentHashMap.put(ConcurrentHashMap.java:1006)  
at interviewBit.main(interviewBit.java:12)
```

As we can observe from the above outputs we can store the null value in the hashmap but it can't be stored in a ConcurrentHashMap as it gives NullPointerException.

8. As we know that hashcodes are generated for each and every key but what happens when the same hashcode is generated for distinct keys?

- When the same hashcode is generated for distinct keys then a collision will occur as the bucket address of the two keys will be the same.
- We have discussed that internally hashmap uses a linked list therefore the linked list will store them all together.

HashMap Coding Interview Questions

1. Write a program to make a hashmap synchronized in java.

`Collections.synchronizedMap()` method is used for making a hashmap synchronized in java.

Here is the implementation:

```
import java.util.Collections;  
import java.util.HashMap;  
import java.util.Map;  
  
public class JavaHashMapPrograms  
{  
    public static void main(String[] args)  
    {  
  
        HashMap<String, Integer> InterviewBit_map = new HashMap<String, Integer>();  
        Map<String, Integer> syncMap = Collections.synchronizedMap(InterviewBit_map);  
    }  
}
```

2. Which method is used for removing or deleting all the key-value pairs? Write a program for removing all the key-value pairs from a HashMap.

Clear() method is used for removing or deleting all the key-value pairs.

Implementation:

```
import java.util.HashMap;

public class InterviewBit
{
    public static void main(String[] args)
    {
        HashMap<String, Integer> InterviewBit_map = new HashMap<String, Integer>();

        InterviewBit_map.put("X", 1);

        InterviewBit_map.put("Y", 2);

        InterviewBit_map.put("Z", 3);

        InterviewBit_map.put("W", 4);

        InterviewBit_map.clear();

        System.out.println(InterviewBit_map.size());    //Output : 0
    }
}
```

3. Design and implement a class which contains two function find() as well as add()

The find function will check whether any pair exists whose sum is equal to a given value in $O(N)$ average time whereas add function will be used for adding any element in $O(1)$ average time.

A hashmap will be the best choice for meeting our requirements as adding an element can be performed in $O(1)$ time whereas searching or finding an element can be performed in $O(N)$ time.

Implementation:

```
public class TwoSum {
    HashMap<Integer, Integer> our_map;

    public TwoSum() {
        our_map = new HashMap<>();
    }

    public void add(int number) {
        our_map.put(number, our_map.getOrDefault(number, 0) + 1);
    }
}
```

```

public boolean find(int data) {
    for (Integer key : our_map.keySet()) {
        int complement = data - key;
        int freq_comp = freq.getDefault(complement, 0);

        if (data - key == key) {
            if (freq_comp >= 2)
                return true;
        } else {
            if (freq_comp >= 1)
                return true;
        }
    }
    return false;
}
}

```

4. Which method is used for finding out the total number of key-value pairs present in a hashmap? Give example.

Size() method is used for finding out the total number of key-value pairs present in a hashmap.

Example:

```

import java.util.HashMap;

public class InterviewBit
{
    public static void main(String[] args)
    {
        HashMap<String, Integer> InterviewBit_map = new HashMap<String, Integer>();

        InterviewBit_map.put("X", 1);

        InterviewBit_map.put("Y", 2);

        InterviewBit_map.put("Z", 3);

        InterviewBit_map.put("W", 4);

        System.out.println(InterviewBit_map.size());    //Output : 4
    }
}

```

5. Which method is used for checking whether a particular key or a value is present in a HashMap or not? Write a program for checking whether a particular key or a value is present in a HashMap or not. If present return true else return false.

ContainsKey() and **ContainsValue()** methods are used for checking whether a particular key or a value is present in a map or not.

```

import java.util.HashMap;
public class InterviewBit
{
    public static void main(String[] args)
    {
        HashMap<String, Integer> InterviewBit_map = new HashMap<String, Integer>();

        InterviewBit_map.put("X", 1);

        InterviewBit_map.put("Y", 2);

        InterviewBit_map.put("Z", 3);

        InterviewBit_map.put("W", 4);

        int value = InterviewBit_map.get("Y");

        System.out.println(InterviewBit_map.containsKey("X")); //Output : true
    }
}

```

6. 4 sum using hashmap: You will be given four arrays arr1, arr2, arr3, and arr4 all of length n, you have to return the count of tuples (i, j, k, l) satisfying the conditions: $0 \leq i, j, k, l < n$ $nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0$

Approach:

- 2 nested loops will be required for implementing this.
- We will start by creating a hashmap
- Next, all possible sums of 2 elements will be calculated and stored in the map using the nested loops we created earlier.
- Now again the nested loop will be used for calculating the sum of 2 elements of the other two arrays.
- Once we find out all the combinations of the sum, we take the complement of that sum and find that in our hashmap.
- If the complement is found then the count of tuples will be increased by 1 every time
- The time complexity will be $O(N^2)$

Implementation:

```

import java.util.*;

class Solution {
    public int fourSumCount(int[] arr1, int[] arr2, int[] arr3, int[] arr4) {
        HashMap<Integer, Integer> AB = new HashMap<>();
        for (int val1 : arr1)
            for (int val2 : arr2)
                AB.put(val1 + val2, AB.getOrDefault(val1 + val2, 0) + 1);

        int count = 0;
        for (int val3 : arr3)

```

```

        for (int val4 : arr4)
            count += AB.getDefault(-val3 - val4, 0);

    return count;
}
}

```

7. Write a program to find the highest frequency of a character in a given string

You will be given a string and you are required to find the character which occurs the most number of times using a hashmap.

Approach:

- First of all, you are required to create a Hashmap.
- After that iterate over the string and check for each and every character whether it is present in the hashmap or not.
- If the character is not present in the hashmap then insert it else increase the frequency of the character.
- Once the hashmap is filled with the frequency of all the characters in the string, loop over the map and checks for the highest frequency character present on the map.

Implementation:

```

import java.io.*;
import java.util.*;

public class InterviewBit {

    public static void main(String[] args) {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        String str = br.readLine();

        //Creating a hashmap
        HashMap<Character, Integer> frequency_map = new HashMap<>();
        for(int i = 0; i < str.length(); i++){
            Character ch = str.charAt(i);
            if(frequency_map.containsKey(ch) == true){
                int old_freq = frequency_map.get(ch);
                int new_freq = old_freq + 1;
                frequency_map.put(ch, new_freq);
            } else {
                frequency_map.put(ch, 1);
            }
        }

        Set<Character> keys = frequency_map.keySet();
        Character c = str.charAt(0);
        for(Character key : keys){
            if(frequency_map.get(key) > frequency_map.get(c)){
                c = key;
            }
        }
    }
}

```

```

    }

    System.out.println(c);
}
}

```

8. Write a program to show how can we retrieve values from a hashmap and state which method is used for it.

The get() method is used for retrieving the values from the hashmap.

Implementation:

```

import java.util.HashMap;
public class InterviewBit
{
    public static void main(String[] args)

    {
        HashMap<String, Integer> InterviewBit_map = new HashMap<String, Integer>();
        InterviewBit_map.put("X", 1);
        InterviewBit_map.put("Y", 2);
        InterviewBit_map.put("Z", 3);
        InterviewBit_map.put("W", 4);
        int value = InterviewBit_map.get("Y");
        System.out.println(value);    //Output : 2
    }
}

```

9. You will be given an unsorted array of integers and a target t. How will you implement a program using a Hashmap to find the position of two numbers whose sum equals target t?

Approach:

- A general approach for solving such kinds of problems is that first, you have to create a map of Integers.
- Then iterate through the given array and subtract the current value in the array from the given target.
- Look for that value in the hashmap simultaneously.
- If the value is found in the map then return the index of these 2 numbers else insert the current element of the array as the key and its index as the value in the Hashmap.

Implementation:

```

class InterviewBit{
    public int[] target(int[] nums, int t) {
        HashMap<Integer, Integer> find_complement = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            if (find_complement.containsKey(t - nums[i]) == true) {

```



```

        return new int[] { find_complement.get(t - nums[i]), i };
    }
    find_complement.put(nums[i], i);
}
return null;
}
}

```

HashMap MCQ Questions

1.

How does the internal structure of a hash bucket get affected by the presence of a huge no. of items.

The bucket does not gets affected

It will convert from LinkedList to BST

It will convert from LinkedList to an ArrayList

None of the above

2.

Predict the output of the following program:

```

import java.util.*;

class Java_Mcqs
{
    public static void main(String args[]) {
        Map<String, String> INTERVIEWBIT_MAP = new HashMap<String, String>();
        INTERVIEWBIT_MAP.put(new String("a"), "JAVA");
        INTERVIEWBIT_MAP.put(new String("a"), "C++");
        System.out.println(INTERVIEWBIT_MAP);
    }
}
{a=Java}
{a=C++}

```

Runtime error

None of the above

3.

Predict the output of the following program:

```

import java.util.*;

class Java_Mcqs
{
    public static void main(String[] args)
    {
        HashMap<Integer, String> InterviewBit_Map = new HashMap<Integer,
String>();
        InterviewBit_Map.put(1, null);
        InterviewBit_Map.put(7, null);
        InterviewBit_Map.put(5, null);
    }
}

```

```

        InterviewBit_Map.put(2, null);
        InterviewBit_Map.put(4, null);

        System.out.println(InterviewBit_Map);
    }
}

```

Runtime error

Compile-time error

{1=null, 2=null, 4=null, 5=null, 7=null}

{1=null}

4.

Predict the output of the following program:

```

import java.util.*;

class Java_Mcqs
{
    public static void main(String args[]) {
        Map<String, String> INTERVIEWBIT_MAP = new HashMap<String, String>();
        boolean isEmpty = INTERVIEWBIT_MAP.isEmpty();
        System.out.println("Our HashMap is empty: " + isEmpty);
    }
}

```

Compile-time error

Our HashMap is empty: false

Our HashMap is empty: true

Runtime error

5.

What happens when we try to insert a key in a HashMap which is already present.

The old key is replaced by the new key

No change occurs in the HashMap

An exception is thrown

None of the above

6.

What is the default load factor of a HashMap in Java?

0.65

0.75

0.85

0.95

7.

Which method is used for inserting a key in a HashMap in Java?

Put

Insert

Get
None of the above
8.

Which of the following statement is true?

A hashmap can not store null values.
Only null values are allowed but null keys can not be stored in a hashmap.
A hashmap can store any number of null keys as well as null values.
A hashmap can store only one null key and any number of null values.
9.

You are given 4 statements. Choose which one of the following statements is correct.

The time complexity for pushing and retrieving the elements in a hashmap in terms of big O notation is $O(1)$.

The time complexity for pushing and retrieving the elements in a hashmap in terms of big O notation is $O(\log(N))$.

The time complexity for pushing and retrieving the elements in a hashmap in terms of big O notation is $O(N)$.

None of the above.

10.

You are given 4 statements. Choose which one of the following statements is incorrect.

Hashmaps in Java store unique keys

HashMaps in Java is similar to unordered_maps in C++

HashMaps in Java have a load factor of 0.75

HashMaps in Java are thread-safe

Did you know that HashMap is one of the most commonly used data structures in Java programming? In fact, it's widely used in software development. That's why understanding HashMap is essential for many programming roles, especially in Java-based positions. So, to help you with your interview preparation, we've put together a list of over 15 **HashMap interview questions** with simple answers.

This guide includes commonly asked questions suitable for beginners as well as those at intermediate and advanced levels.

Basic HashMap Interview Questions for Freshers

Here are some important **HashMap in java interview questions** and answers for freshers.

1. What is a HashMap in Java?

A HashMap in Java is a data structure that stores key-value pairs. It allows efficient retrieval of values based on unique keys.

When you add an entry to a HashMap using `put(key, value)`, the HashMap computes a hash code for the key to determine where to store the corresponding value.

HashMaps are widely used for fast lookup and mapping operations in Java applications.

2. Can a HashMap contain duplicate values?

Yes, a HashMap can contain duplicate values but not duplicate keys. Each key in a HashMap must be unique, but different keys can map to the same value.

3. What happens if you try to add a duplicate key to a HashMap?

If you attempt to add a duplicate key to a HashMap, the existing value associated with that key will be overwritten by the new value.

4. Why is the initial capacity of a HashMap important?

The initial capacity of a HashMap is important because it affects its performance. Setting an appropriate initial capacity can reduce the number of rehashing operations, which can be costly in terms of time and memory.

5. How do you add elements to a HashMap?

Elements can be added to a HashMap using the `put()` method.

For example:

```
HashMap<String, Integer> map = new HashMap<>();
```

```
map.put("apple", 10);
```

```
map.put("orange", 15);
```

HashMap in Java Interview Questions for Experienced

These are some commonly asked **HashMap** interview questions and answers for experienced candidates.

6. **How does HashMap handle collisions?**

HashMap handles collisions by using a technique called chaining. If multiple keys hash to the same index, their entries are stored as a linked list at that index.

7. **What is the maximum number of entries you can store in HashMap?**

HashMap does not have a fixed maximum limit on entries. When a bucket is full due to hash collisions, HashMap uses a [linked list](#) to manage the keys. This linked list can grow indefinitely, utilizing available memory effectively without a predefined entry limit.

8. **What is the significance of the load factor in a HashMap?**

The load factor determines when the HashMap will resize itself. When the number of elements exceeds the product of the load factor and the current capacity, the HashMap will rehash its contents to a larger capacity, improving performance.

9. **How can you make a custom object usable as a key in a HashMap?**

To use a custom object as a key in a HashMap, you must override the `hashCode()` and `equals()` methods in the object class. This ensures that HashMap can compute the hash code correctly and check for equality between keys.

10. **What factors determine the performance of a HashMap?**

The performance efficiency of a HashMap depends on two key factors:

- **Capacity:** Capacity refers to the total number of buckets in a HashMap. By default, a HashMap starts with a capacity of sixteen buckets. As more key-value pairs are added to the HashMap, its capacity can be increased dynamically.
- **Load Factor:** The load factor determines when the HashMap should increase its capacity. The default load factor is 75%. When the number of entries exceeds 75% of the current capacity, the HashMap automatically increases its capacity to maintain performance efficiency.

Advanced HashMap and HashTable Interview Questions

These are some advanced **HashMap** interview questions, along with answers.

11. **What is the difference between HashMap and HashTable in Java?**

HashMap is not synchronized and is not thread-safe, whereas HashTable is synchronized and is thread-safe. Additionally, HashMap allows null keys and values, whereas HashTable does not allow null keys or values.

12. **How does HashMap handle concurrency issues, and why might ConcurrentHashMap be preferred in concurrent environments?**

HashMap is not thread-safe and can lead to concurrent modification exceptions in a multi-threaded environment. ConcurrentHashMap, on the other hand, uses finer-grained locking mechanisms to achieve thread safety, making it more suitable for concurrent applications.

13. **What happens if two different keys have the same hash code in a HashMap?**

If two different keys have the same hash code (hash collision), HashMap uses a linked list (or a balanced tree in [Java 8+](#)) to store these keys and their corresponding values within the same bucket.

14. **How can you improve the performance of HashMap in terms of reducing collisions?**

To reduce collisions in HashMap, you can provide custom implementations of hashCode() and equals() methods in key objects to distribute keys more evenly across buckets, thus improving performance.

Internal Working of HashMap Interview Questions

Here are some important **HashMap in java interview questions** and answers.

15. **How does HashMap store key-value pairs internally?**

HashMap uses an array of buckets (or slots) to store key-value pairs. Each bucket can hold multiple entries due to potential hash collisions.

16. **What is hashing, and why is it important in HashMap?**

Hashing is a technique used by HashMap to convert a key into an index (hash code) for efficient storage and retrieval. It's important because it allows HashMap to quickly locate the bucket where a key-value pair should be stored or retrieved.

17. **What is the role of the hashCode() method in HashMap?**

The hashCode() method is used by HashMap to compute the hash code of keys. This hash code determines the index (bucket) where the key-value pair will be stored and helps in efficient retrieval.

18. **Why are keys in a HashMap typically immutable objects?**

Keys in HashMap are typically immutable to ensure that their hash code remains consistent throughout their lifetime. This prevents unexpected behaviour, such as not being able to retrieve values due to a change in hash code.

HashMap Coding Questions

Here are some common **HashMap programming interview questions** and answers.

19. How do you check if a key exists in a HashMap?

You can use the `containsKey()` method to check if a specific key exists in a HashMap.

For example:

```
HashMap<String, Integer> map = new HashMap<>();

map.put("apple", 10);

if (map.containsKey("apple")) {

    System.out.println("Key 'apple' exists in the HashMap!");

}
```

20. How do you retrieve a value from a HashMap given a key?

Use the `get()` method to retrieve the value associated with a given key in a HashMap. **For example:**

```
int value = map.get("apple");

System.out.println("Value associated with 'apple': " + value);
```

21. How do you remove a key-value pair from a HashMap?

You can use the `remove()` method to remove a specific key-value pair from a HashMap.

For example:

```
map.remove("apple");
```

22. How do you check if a HashMap is empty?

You can use the `isEmpty()` method to check if a HashMap contains any key-value pairs.

For example:

```
if (map.isEmpty()) {

    System.out.println("HashMap is empty!");

} else {

    System.out.println("HashMap is not empty!");

}
```